

— INFORME TECNICO DE CIBERSEGURIDAD Y ARQUITECTURA CLOUD

AUDITORIA DE SEGURIDAD LA RAGAZZA WEB

Analisis tecnico integral para la materia Lenguajes de Internet: autenticacion, manejo de credenciales, sentencias preparadas equivalentes, XSS hardening, arquitectura cloud/local, matriz CIA, riesgos y migracion.

ESTUDIANTE

Juan Guillermo López Cortés

MATERIA

Lenguajes de Internet

PROYECTO

La Ragazza Web

FECHA

2026-05-23



CREDENCIALES DELEGADAS

Supabase Auth gestiona passwords y hashes internos; la app no guarda Contraseñas propias.



QUERY BUILDER

Equivalente operacional a prepared statements: no hay SQL interpolado por la app.



DEFENSA XSS

Rechazo de markup persistente en backend y escapeHtml/textContent en frontend.



MATRIZ CIA

Activos clasificados por confidencialidad, integridad, disponibilidad y riesgo residual.

DOCUMENTO INTEGRADO

INFORME PREMIUM DE AUDITORIA Y ARQUITECTURA

Este HTML integra el contenido completo de docs/security-audit-report.md y todos los diagramas ubicados en docs/diagrams/. Esta version esta preparada para lectura academica, revision empresarial y exportacion a PDF desde el navegador.

Proyecto: **La Ragazza Web** Fecha del informe: **2026-05-23** Alcance: sitio Astro, panel administrativo, API Parcial Express, Supabase Auth y Supabase/Postgres.

1. RESUMEN EJECUTIVO

Este informe documenta dos controles de seguridad obligatorios para la rubrica:

1. **Proteccion de credenciales:** la aplicacion no almacena Contraseñas en tablas propias. El login, signup, sesiones y almacenamiento de hashes de Contraseña son gestionados por **Supabase Auth**.
2. **Prevencion de inyeccion SQL:** el backend no construye SQL crudo con datos del usuario. La capa de repositorios usa el **Supabase Query Builder**, que actua como equivalente practico de sentencias preparadas en esta arquitectura.

Tambien se incluye:

- analisis de arquitectura cloud y local;
- matriz CIA;
- analisis de riesgos;
- evidencias de codigo;
- guía para capturas de pantalla;
- manual de migracion;
- referencias a diagramas HTML separados.

Diagramas complementarios:

- `docs/diagrams/security-cloud-architecture.html`
- `docs/diagrams/security-local-architecture.html`
- `docs/diagrams/admin-security-flow.html`
- `docs/diagrams/cia-security-matrix.html`

2. ALCANCE TECNICO DEL SISTEMA

2.1 COMPONENTES PRINCIPALES

COMPONENTE	TECNOLOGIA	RESPONSABILIDAD
Sitio publico	Astro	Renderizar paginas estaticas bilingues para usuarios finales.
Panel admin	Astro + JS publico en <code>public/admin</code>	Permitir login, gestion de reservas y solicitudes de cambio de rol.
Autenticacion	Supabase Auth	Crear usuarios, autenticar credenciales, emitir JWT y manejar sesiones.
API Parcial	Express	Validar JWT, aplicar reglas de negocio y consultar Supabase/Postgres.
Base de datos	Supabase/Postgres	Persistir perfiles, reservas y solicitudes con RLS.
Hosting frontend	Vercel	Servir sitio estatico y panel admin.
Hosting API	Render u otro host Node	Ejecutar Express API.

2.2 FLUJO CLOUD DESPLEGADO

1. El usuario abre el panel admin desde Vercel.
2. El navegador carga el cliente Supabase publico con la `anon/publishable key`.
3. El usuario inicia sesion desde el formulario del admin.
4. Supabase Auth valida email y Contraseña.
5. Supabase Auth devuelve una sesion y un `access_token`.
6. El admin llama a la API Parcial con `Authorization: Bearer <access_token>`.
7. La API valida el token con Supabase.
8. La API crea un cliente Supabase server-side con el JWT del usuario.
9. Los servicios validan el payload.
10. Los repositorios consultan con Supabase Query Builder.
11. Supabase/Postgres aplica RLS.
12. La API devuelve JSON al admin.

2.3 FLUJO LOCAL DE DESARROLLO

1. Astro corre en `localhost:4321`.
2. Express API corre en `localhost:3001`.
3. `astro.config.mjs` proxya `/api` hacia `http://localhost:3001`.
4. El admin local usa Supabase Auth real o el proyecto Supabase configurado en `.env`.
5. El flujo de JWT y RLS es el mismo que en cloud.

3. PROTECCION DE CONTRASEÑAS

3.1 DECISION DE ARQUITECTURA

La aplicacion **no implementa almacenamiento propio de Contraseñas**. No existe una tabla de aplicacion como:

- `users.password`
- `profiles.password`
- `admin_users.password`
- `password_hash`

La tabla `profiles` solo extiende la identidad del usuario con informacion de dominio, por ejemplo:

- `id`
- `email`
- `role`

La Contraseña no forma parte del modelo de datos de la aplicacion.

3.2 DONDE SE AUTENTICA EL USUARIO

El formulario de login/signup del admin usa Supabase Auth desde el navegador. La evidencia esta en:

- `public/scripts/admin/auth.js`

Fragmento relevante:

```
const { error } = await supabase.auth.signUp({
  email: String(payload.email ?? "").trim(),
  password: String(payload.password ?? "")
});
```

```
const { error } = await supabase.auth.signInWithPassword({
  email: String(payload.email ?? "").trim(),
  password: String(payload.password ?? "")
});
```

Interpretacion:

- El frontend envia las credenciales a Supabase Auth.
- La API Parcial no recibe Contraseñas.
- La API Parcial no guarda Contraseñas.
- La base de datos de aplicacion no guarda Contraseñas.
- Supabase Auth gestiona internamente los hashes y el ciclo de vida de credenciales.

3.3 DONDE SE VALIDA LA SESION EN EL BACKEND

La API no valida email/Contraseña. Solo valida tokens ya emitidos por Supabase Auth.

Archivo:

- `parcial/api/src/lib/supabase.mjs`

Fragmento:

```
export function getBearerToken(request) {
  const header = request.headers.authorization ?? "";
  if (!header.toLowerCase().startsWith("bearer ")) return "";
  return header.slice(7).trim();
}
```

```
export async function resolveRequestAuth(request) {
  const accessToken = getBearerToken(request);
  if (!accessToken) {
```

```

    throw new HttpError(401, "Sesion requerida.");
  }

  const supabase = createServerSupabaseClient(accessToken);
  const { data: userData, error: userError } = await supabase.auth.getUser();
  if (userError || !userData.user) {
    throw new HttpError(401, "Sesion requerida.");
  }
}

```

Interpretacion:

- El backend solo acepta **Bearer token**.
- El token se valida contra Supabase Auth.
- Si el token no existe o no es valido, la API responde **401**.
- La API nunca compara Contraseñas.

3.4 CAPTURA DE PANTALLA SOLICITADA: CONTRASEÑAS CIFRADAS EN BD

La rubrica solicita: "**Captura de pantalla de las Contraseñas cifradas en la BD.**"

En este proyecto se debe presentar de esta forma:

1. Abrir Supabase Dashboard.
2. Ir al proyecto usado por la aplicacion.
3. Entrar a **Authentication**.
4. Abrir la seccion de usuarios.
5. Mostrar que los usuarios son administrados por Supabase Auth.
6. Si se usa SQL Editor con permisos adecuados, consultar la tabla interna **auth.users**.
7. Evidenciar que la Contraseña no aparece en texto plano y que Supabase maneja un campo interno de hash/encrypted password.

Consulta de evidencia sugerida para Supabase SQL Editor:

```

select
  id,

```

```
email,  
encrypted_password,  
created_at,  
last_sign_in_at  
from auth.users  
limit 5;
```

Nota importante: esta tabla pertenece al esquema interno `auth`, no a las tablas de aplicacion. La captura debe ocultar o difuminar valores sensibles completos. Para la sustentacion basta mostrar que existe un valor cifrado/hasheado y que no hay Contraseñas en texto plano.

3.5 CAPTURA ALTERNATIVA ACEPTABLE

Si Supabase no permite visualizar `encrypted_password` por permisos del dashboard, se recomienda incluir dos capturas:

1. Pantalla de **Authentication > Users** en Supabase, mostrando que los usuarios viven en Supabase Auth.
2. Pantalla del schema o tabla `public.profiles`, mostrando que solo guarda `id`, `email` y `role`, no `password`.

Conclusion para la rubrica:

La aplicacion no almacena Contraseñas en su propia base de datos. Las credenciales son delegadas a Supabase Auth, que administra internamente el hash de Contraseña en el esquema `auth`. La aplicacion solo consume JWTs emitidos por Supabase y guarda datos de perfil/rol en `public.profiles`.

4. PRUEBA DE SENTENCIAS PREPARADAS

4.1 ACLARACION TECNICA

El requisito habla de "sentencias preparadas". En esta arquitectura no se usa un driver SQL directo como `pg`, `PDO` o `mysqli`. El backend usa Supabase JS SDK.

Por eso, la equivalencia tecnica es:

REQUISITO TRADICIONAL	IMPLEMENTACION EN ESTE PROYECTO
Sentencia preparada	Supabase Query Builder
Parametros separados del SQL	Valores pasados como argumentos/objetos a <code>.eq()</code> , <code>.insert()</code> , <code>.update()</code>
No concatenar SQL	No existe SQL crudo construido con input
Control de permisos	RLS en Supabase/Postgres

La razon de seguridad es la misma: **la entrada del usuario no se concatena en un string SQL ejecutable.**

4.2 CAPA DONDE SE IMPLEMENTA

Las consultas equivalentes a preparadas se implementan en la capa de repositorios:

- `parcial/api/src/repositories/bookingRepository.mjs`
- `parcial/api/src/repositories/roleRequestRepository.mjs`

La capa de servicios valida datos, pero no consulta directamente la base. La capa de rutas recibe HTTP, pero tampoco construye SQL. La responsabilidad de acceso a datos vive en repositorios.

4.3 EVIDENCIA EN `bookingRepository.mjs`

4.3.1 INSERT SEGURO DE RESERVAS

Archivo:

- `parcial/api/src/repositories/bookingRepository.mjs`

Lineas relevantes:

- 35 a 49

Codigo:

```

async create(supabase, { userId, nombreCliente, bookingTime, numeroPersonas
  const data = await runQuery(
    supabase
      .from("bookings")
      .insert({
        user_id: userId,
        client_name: nombreCliente,
        booking_time: bookingTime,
        party_size: numeroPersonas,
        comments: comentarios,
        status
      })
      .select(BOOKING_SELECT)
      .single()
  );

  return mapBooking(data);
}

```

Por que es seguro:

- `nombreCliente`, `comentarios`, `bookingTime` y demas valores entran como propiedades de un objeto.
- No existe una cadena `INSERT INTO ... VALUES ('${nombreCliente}')`.
- Supabase/PostgREST recibe estructura y valores por separado.
- Antes de llegar aqui, `bookingService` valida formato, longitud y markup HTML.

4.3.2 SELECT SEGURO POR USUARIO

Lineas relevantes:

- 54 a 59

Codigo:

```

async listByUserId(supabase, userId) {
  const data = await runQuery(
    supabase.from("bookings").select(BOOKING_SELECT).eq("user_id", userId).
    ascending: true
  );
}

```

```

    })
  );

  return data.map(mapBooking);
}

```

Por que es seguro:

- `userId` se pasa como argumento de `.eq("user_id", userId)`.
- No se concatena en un `WHERE user_id = '${userId}'`.
- La consulta tambien se ejecuta bajo el contexto del JWT del usuario, por lo que RLS limita el acceso.

4.3.3 SELECT SEGURO POR ID

Lineas relevantes:

- 74 a 76

Codigo:

```

async findById(supabase, id) {
  const data = await runQuery(supabase.from("bookings").select(BOOKING_SELECT));
  return mapBooking(data);
}

```

Por que es seguro:

- `id` se pasa como valor parametrizado del query builder.
- No existe concatenacion SQL.
- `maybeSingle()` limita la expectativa de resultado a cero o un registro.

4.3.4 UPDATE SEGURO DE RESERVAS

Lineas relevantes:

- 79 a 87

Codigo:

```

async update(supabase, id, payload) {
  const data = await runQuery(
    supabase
      .from("bookings")
      .update(payload)
      .eq("id", id)
      .select(BOOKING_SELECT)
      .single()
  );

  return mapBooking(data);
}

```

Por que es seguro:

- `payload` es un objeto validado previamente por el servicio.
- `id` entra por `.eq("id", id)`.
- No se arma `UPDATE bookings SET ... WHERE id = ...` manualmente.

4.3.5 DELETE SEGURO DE RESERVAS

Lineas relevantes:

- 96 a 99

Codigo:

```

async remove(supabase, id) {
  const { error } = await supabase.from("bookings").delete().eq("id", id);
  if (error) throw error;
}

```

Por que es seguro:

- El filtro de borrado usa `.eq("id", id)`.
- No se concatena el ID dentro de SQL textual.

4.4 EVIDENCIA EN `roleRequestRepository.mjs`

4.4.1 INSERT SEGURO DE SOLICITUDES

Lineas relevantes:

- 40 a 47

Codigo:

```
async create(supabase, { userId, requestedRole, justification }) {
  const data = await runQuery(
    supabase
      .from("role_change_requests")
      .insert({ user_id: userId, requested_role: requestedRole, justificati
      .select(REQUEST_SELECT)
      .single()
  );

  const requesterEmail = await this.findRequesterEmail(supabase, data.user_
  return mapRequest(data, requesterEmail);
}
```

Por que es seguro:

- `justification` puede venir del usuario, pero viaja como valor de objeto.
- `requestedRole` ya fue validado contra un conjunto permitido.
- No existe SQL crudo con interpolacion.

4.4.2 SELECT SEGURO CON `.in()`

Lineas relevantes:

- 29 a 36

Codigo:

```
const userIds = [...new Set(requests.map((request) => request.user_id))];
const profiles = await runQuery(supabase.from("profiles").select("id,email"
```

Por que es seguro:

- La lista `userIds` se pasa como arreglo a `.in("id", userIds)`.
- No se construye un `IN ( ${ids.join(",")} )` manual.
- Supabase maneja la parametrización del filtro.

4.4.3 SELECT SEGURO POR USUARIO

Lineas relevantes:

- 61 a 66

Código:

```
async listByUser(supabase, userId) {
  const data = await runQuery(
    supabase.from("role_change_requests").select(REQUEST_SELECT).eq("user_id", userId).order("created_at", {
      ascending: false
    })
  );

  return enrichWithRequesterEmails(supabase, data);
}
```

Por que es seguro:

- `userId` entra por `.eq()`.
- No se interpola dentro de un string SQL.

4.4.4 UPDATE SEGURO DE SOLICITUD

Lineas relevantes:

- 79 a 87

Código:

```
async update(supabase, id, { requestedRole, justification }) {
  const data = await runQuery(
    supabase
      .from("role_change_requests")
      .update({ requested_role: requestedRole, justification })
  );
}
```

```

        .eq("id", id)
        .select(REQUEST_SELECT)
        .single()
    );

    const requesterEmail = await this.findRequesterEmail(supabase, data.user_
    return mapRequest(data, requesterEmail);
}

```

Por que es seguro:

- `requestedRole` y `justification` se envian como objeto.
- `id` se envia por `.eq()`.
- La capa de servicio ya rechaza roles invalidos, textos demasiado largos y markup HTML.

4.4.5 UPDATE SEGURO DE ESTADO

Lineas relevantes:

- 93 a 101

Codigo:

```

async updateStatus(supabase, id, status) {
    const data = await runQuery(
        supabase
            .from("role_change_requests")
            .update({ status })
            .eq("id", id)
            .select(REQUEST_SELECT)
            .single()
    );
}

```

Por que es seguro:

- `status` viene de un conjunto permitido.
- `id` se filtra por `.eq()`.
- No se concatena SQL.

4.4.6 UPDATE SEGURO DE ROLES CON SERVICE CLIENT

Lineas relevantes:

- 107 a 136

Codigo resumido:

```
const { data, error } = await supabase
  .from("profiles")
  .update({ role })
  .eq("id", userId)
  .select("id,role")
  .maybeSingle();
```

Y fallback:

```
const service = createServiceSupabaseClient();
const { data: svcData, error: svcError } = await service
  .from("profiles")
  .update({ role })
  .eq("id", userId)
  .select("id,role")
  .maybeSingle();
```

Por que es seguro:

- Incluso en operacion privilegiada se usa Query Builder.
- No hay SQL crudo.
- El `service role key` solo se usa server-side y en un flujo auditado.
- El rol aprobado viene de solicitudes validadas y de un conjunto permitido.

4.5 CAPA PREVIA DE VALIDACION

La seguridad de consultas se complementa con validacion de dominio.

RESERVAS

Archivo:

- `parcial/api/src/services/bookingService.mjs`

Lineas relevantes:

- 7 a 18: normalizacion.
- 21 a 50: validacion.
- 72 a 80: creacion tras validar.
- 83 a 107: actualizacion tras validar.

Controles:

- campos obligatorios;
- numero de personas entero y mayor a cero;
- comentarios maximo 800 caracteres;
- nombre maximo 120 caracteres;
- fecha/hora validas;
- reserva no puede estar en el pasado;
- rechazo de `<` y `>` en campos libres.

SOLICITUDES DE ROL

Archivo:

- `parcial/api/src/services/roleRequestService.mjs`

Lineas relevantes:

- 21 a 43: validacion de payload.
- 59 a 66: creacion tras validar.
- 69 a 79: actualizacion tras validar.

Controles:

- rol solicitado obligatorio;
- justificacion obligatoria;
- justificacion minimo 12 y maximo 1200 caracteres;
- rol solicitado solo puede ser `customer`, `staff` o `admin`;
- rechazo de `<` y `>` en justificacion.

5. MATRIZ CIA

ACTIVO	CONFIDENCIALIDAD	INTEGRIDAD	DISPONIBILIDAD	CONTROLES IMPLEMENTADOS
Contraseñas de usuarios	Alta	Alta	Media	Delegadas a Supabase Auth; no se guardan en tablas propias; Supabase almacena hashes internos.
JWT de sesion	Alta	Alta	Media	Enviado como Bearer token; validado con <code>supabase.auth.getUser</code> ; no se almacena en backend.
Perfiles (<code>profiles</code>)	Media	Alta	Media	RLS, validacion de token, acceso por Supabase client contextual.
Reservas (<code>bookings</code>)	Media	Alta	Alta	RLS, validacion de servicio, Query Builder, controles de rol.
Solicitudes de rol	Media	Alta	Media	RLS, validacion de rol, estados permitidos, Query Builder.
Service role key	Critica	Critica	Media	Solo server-side; nunca en frontend; usada en flujo auditado de aprobacion de rol.
Panel admin	Media	Alta	Alta	Supabase Auth, validacion de sesion, escape HTML, CORS controlado.
API Parcial	Media	Alta	Alta	Helmet, CORS, <code>requireAuth</code> , <code>HttpError</code> , validacion, tests.

LECTURA DE LA MATRIZ

- **Confidencialidad:** se protege principalmente con Supabase Auth, JWT, RLS y no exposicion del service key.
- **Integridad:** se protege con validacion de servicios, Query Builder, reglas de rol y RLS.
- **Disponibilidad:** depende de Vercel, Render y Supabase; el diseno separa frontend estatico y API para reducir acoplamiento.

6. ANALISIS DE RIESGOS

RIESGO	PROBABILIDAD	IMPACTO	ESTADO	MITIGACION
Robo de Contraseñas desde tabla propia	Baja	Critico	Mitigado	No hay tabla propia de Contraseñas; Supabase Auth gestiona hashes.
SQL injection en filtros <code>id / user_id</code>	Baja	Alto	Mitigado	<code>.eq()</code> , <code>.in()</code> , <code>.insert()</code> , <code>.update()</code> con Query Builder.
SQL injection en comentarios/justificaciones	Baja	Alto	Mitigado	Valores viajan como objeto; servicio valida longitud y markup.
XSS almacenado en comentarios	Baja	Alto	Mitigado	Backend rechaza <code></></code> ; frontend aplica <code>escapeHtml()</code> .
XSS reflejado en mensajes de error	Baja	Medio	Mitigado	Mensajes se muestran con <code>textContent</code> .
Exposicion de service role key	Media	Critico	Control operativo	Debe existir solo en Render/API, nunca en Vercel/frontend.
CORS bloqueando API o permitiendo origen indebido	Media	Medio	Mitigado parcialmente	Middleware CORS permite origenes conocidos; revisar <code>CORS_ALLOWED_ORIGINS</code> al migrar.

RIESGO	PROBABILIDAD	IMPACTO	ESTADO	MITIGACION
Usuario sin perfil tras signup	Media	Medio	Requiere operacion	Trigger <code>handle_new_user()</code> o provisionamiento manual de <code>profiles</code> .

7. MANUAL DE MIGRACION DETALLADO

7.1 VARIABLES DE ENTORNO

Frontend Vercel:

```
SUPABASE_URL=https://<project>.supabase.co
SUPABASE_ANON_KEY=<publishable-or-anon-key>
PUBLIC_API_BASE_URL=https://<api-host>
```

API Render/local:

```
NODE_ENV=production
PORT=3001
SUPABASE_URL=https://<project>.supabase.co
SUPABASE_ANON_KEY=<publishable-or-anon-key>
SUPABASE_SERVICE_ROLE_KEY=<service-role-key>
PUBLIC_SITE_URL=https://<frontend-host>
CORS_ALLOWED_ORIGINS=https://<frontend-host>
```

Regla critica:

- `SUPABASE_SERVICE_ROLE_KEY` nunca debe existir en el frontend.
- `SUPABASE_ANON_KEY` o `sb_publishable_...` si puede estar en el navegador; no es una private key.



7.2 MIGRACION LOCAL

1. Instalar dependencias:

```
pnpm install
```

1. Configurar `.env` local con Supabase URL y anon key.
2. Ejecutar API y frontend:

```
pnpm dev
```

1. Abrir:

```
http://localhost:4321/admin
```

1. Probar login con Supabase Auth.
2. Probar reservas y solicitudes.
3. Ejecutar pruebas:

```
pnpm parcial:test  
pnpm build
```

7.3 MIGRACION CLOUD

1. Crear o verificar proyecto Supabase.
2. Configurar Auth con email/password.
3. Crear tablas publicas necesarias:
 - `profiles` - `bookings` - `role_change_requests`
1. Habilitar RLS.
2. Crear politicas de acceso.
3. Configurar trigger de perfil si aplica.
4. Desplegar API en Render.
5. Configurar variables de entorno de API.
6. Desplegar frontend en Vercel.
7. Configurar `PUBLIC_API_BASE_URL` apuntando a Render.

- Configurar `CORS_ALLOWED_ORIGINS` en API para permitir el dominio Vercel.
- Probar preflight CORS desde navegador.
- Probar login y CRUD.
- Ejecutar smoke tests si el entorno lo permite.

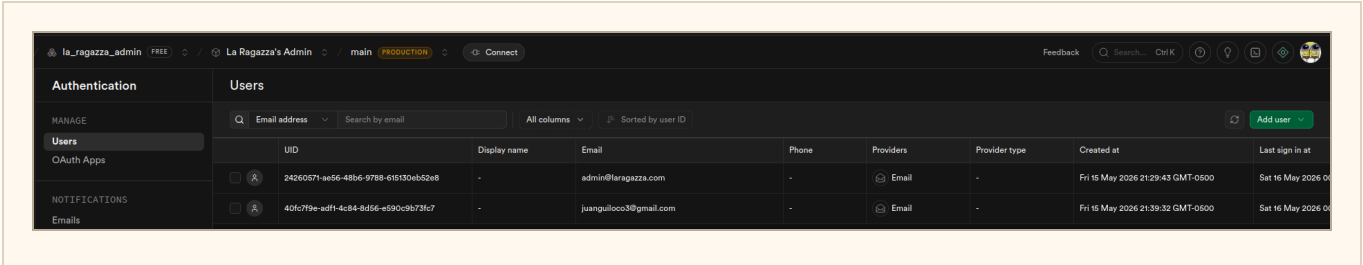
7.4 CHECKLIST POST-MIGRACION

VERIFICACION	RESULTADO ESPERADO
Login admin	Supabase emite sesion.
API sin token	Responde 401.
API con token valido	Responde datos permitidos por RLS.
Preflight CORS	<code>OPTIONS</code> responde 204 para origen permitido.
Payload <code><script></code> en reserva	API responde 400.
Payload <code><script></code> en solicitud	API responde 400.
<code>profiles</code>	No contiene columna password.
<code>auth.users</code>	Contiene hash interno gestionado por Supabase.
Service key	Solo en API host.

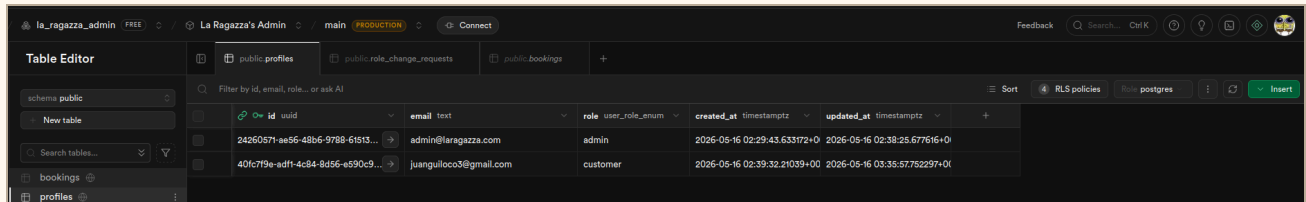
8. EVIDENCIAS ANEXADAS Y ENCONTRADAS EN EL REPO DEL PROYECTO

8.1 CAPTURAS

Las dos capturas de Supabase Auth ya estan incorporadas aqui.



Supabase Authentication Users. La cuenta se gestiona en Supabase Auth y no en una tabla propia con contraseña.



id	email	role	created_at	updated_at
24260571-ae56-48b5-9788-61513...	admin@laragazza.com	admin	2026-05-16 02:29:43.633172+00	2026-05-16 02:38:25.677616+00
40fc7f9e-adf1-4c84-8d56-a590c9...	juanguloco3@gmail.com	customer	2026-05-16 02:39:32.21039+00	2026-05-16 03:35:57.782297+00

Tabla public.profiles sin columna de Contraseña. Solo se conservan datos de perfil y rol.

- Código `public/scripts/admin/auth.js` usando `signInWithPassword`.
- Código `bookingRepository.mjs` usando `.insert()`, `.eq()`, `.update()`.
- Código `roleRequestRepository.mjs` usando `.insert()`, `.eq()`, `.in()`, `.update()`.

8.2 COMANDOS DE EVIDENCIA

Buscar SQL crudo:

```
rg -n "query\\(|execute\\(|raw\\(|sql`|SELECT|INSERT|UPDATE|DELETE" parcial
```

Ejecutar pruebas:

```
pnpm parcial:test
```

Construir el sitio:

```
pnpm build
```

9. CONCLUSION

El proyecto cumple el objetivo de la rubrica desde la logica de seguridad:

- Las Contraseñas no se almacenan en tablas propias; se delegan a Supabase Auth.

- La API no implementa comparacion de Contraseñas; solo valida JWTs.
- Las consultas a la base no se escriben como SQL interpolado; se hacen con Supabase Query Builder.
- Los valores del usuario se pasan como parametros/objetos a `.insert()`, `.update()`, `.eq()` e `.in()`.
- La validacion de servicios reduce datos invalidos antes de persistir.
- RLS limita el acceso a filas segun identidad.
- La arquitectura cloud separa frontend estatico, API y proveedor de identidad/base de datos, lo que permite controles por capa.